

Friedrich-Alexander-Universität
Erlangen-Nürnberg (FAU)

NHR@FAU

Assignment 0: Warming Up

Programming Techniques for Supercomputers (PTfS)

Muhammad Omar
Student IdM: to37gulo
Student HPC-Account: ptfs356v

Gehan Younes
Student IdM: qi39dyry
Student HPC-Account: ptfs437v

April 29, 2026

1. Numerical Integration Benchmark and Timing (30 Credits)

Goal

Approximate the integral

$$\int_0^1 4\sqrt{1-x^2}\,dx \approx \pi \approx 3.1415926...$$

using the midpoint rule, and measure the runtime of the loop with high-resolution timing on the Fritz cluster.

Method

- Midpoint rule numerical integration.
- Runtime measured with `CLOCK_MONOTONIC`.
- Loop exits when runtime exceeds 10^{-7} seconds.
- Fixed CPU frequency using SLURM's `-cpu-freq` option.

Compiler and Execution Details

- **Compiler:** Intel `icx`
- **Flags:** `-O3 -xHost`
- **Submission node:** `fritz1`
- **Compute node:** `f0301`
- **Run command:**

```
srun --cpu-freq=2400000-2400000:performance ./pi_bench
```

Output

```
pi = 3.1524114333 with N = 10
Runtime = 0.000000149 seconds
```

Code Snippet

```
#include <stdio.h>
#include <math.h>
#include <time.h>

double getTimeStamp() {
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return (double)ts.tv_sec + (double)ts.tv_nsec * 1e-9;
}

int main() {
    double sum, x, delta_x, S, E;
    int N = 10;

    do {
```

```
        S = getTimeStamp();
        delta_x = 1.0 / N;
        sum = 0.0;
        for (int i = 0; i < N; i++) {
            x = (i + 0.5) * delta_x;
            sum = sum + 4.0 * sqrt(1.0 - x * x);
        }
        double result = sum * delta_x;
        E = getTimeStamp();
        if (E - S > 1e-7) {
            printf("N         = %d\n", N);
            printf("Result    = %.10f\n", result);
            printf("Runtime   = %.6f seconds\n", E - S);
            break;
        }
        N *= 2;
    } while (1);

    return 0;
}
```

2. CPU Cycles per Iteration and Timing Threshold (30 Credits)

Goal

Determine a reasonable minimum runtime to obtain stable and reproducible performance measurements, and compute the number of CPU cycles required per loop iteration.

Approach

I modified the threshold for minimum runtime by changing the condition in the benchmark from `1e-7` seconds to higher values, and observed the output, runtime, and per-iteration performance for each case.

All runs were executed on the Fritz compute node using a fixed CPU frequency of 2.4 GHz via:

```
srun --cpu-freq=2400000-2400000:performance ./pi_bench
```

Results

Using the formula:

$$\text{cycles/iter} = \frac{t \times f_{\text{clock}}}{N} \tag{1}$$

with $f_{\text{clock}} = 2.4 \times 10^9$ Hz, the cycles per iteration for each threshold are calculated as follows:

Threshold (s)	N	Result	Runtime (s)	Time per iteration (ns)	Cycles/Iter
10^{-6}	1,280	3.1416001745	0.000002	1.563	3.75
10^{-5}	10,240	3.1415929860	0.000013	1.270	3.05
10^{-4}	81,920	3.1415926683	0.000103	1.257	3.02

Table 1: Pi integration benchmark results at 2.4 GHz fixed clock frequency.

Analysis and Conclusion

From the results above, we observe that as the threshold increases:

- The per-iteration time becomes more stable.
- The estimated CPU cycles converge toward a consistent value around 3.0 cycles per iteration.

Given the relatively small number of cycles per iteration, measurements with thresholds below 10^{-6} seconds are noisy and unreliable. Therefore, a minimum threshold of $\mathbf{1 \times 10^{-4}}$ seconds is recommended to ensure accurate and reproducible performance analysis.

The result of 3 cycles per iteration is reasonable. Without any optimization, the 'sqrt' operation alone would cost 20 cycles per iteration on Ice Lake. However, with '-O3 -xHost', the Intel compiler automatically vectorizes the loop using AVX-512 instructions, processing 8 doubles at once via 'vsqrtpd', and pipelines multiple iterations — effectively amortizing the 12-cycle sqrt throughput over 8 iterations, giving an ideal of 1.5 cycles/iter. The measured 3 cycles/iter sits slightly above this ideal due to loop overhead (the multiply, subtract, and accumulate operations), which is exactly what one would expect from a well-optimized but not hand-tuned vectorized loop.

3. Performance Metrics and Discussion (10 Credits)

Calculations are based on the results obtained with a threshold of 10^{-4} s.

Performance Calculation

Assuming 7 floating-point operations per iteration (1 sqrt, 3 additions, and 3 multiplications), the total FLOPs is:

$$\text{Total FLOPs} = 7 \times N$$

With $N = 81920$ and elapsed time $T = 1.03 \times 10^{-6}$ seconds, we calculate:

$$\text{FLOPs/s} = \frac{7 \times 81920}{1.03 \times 10^{-6}} \approx 5.567\,\text{GFlop/s}$$

Is FLOPs/s a Good Metric Here?

Flop/s is a common performance metric in HPC, but in this case, it is not ideal. This code performs only a small number of floating-point operations per iteration, with most time spent in a single square root, which is slow and difficult to optimize.

As a result, the FLOP/s metric gives an inflated sense of performance and does not reflect the efficiency or computational cost of the loop.

Alternative Metrics

More appropriate metrics include:

- **Cycles per iteration:** A direct hardware-level measure of how many CPU cycles are needed per loop iteration. In our case, this was consistently ≈ 3 cycles.
- **Time per iteration:** A useful latency-focused metric for small loops, which was $\approx 1.2 \times 10^{-9}$ seconds per iteration.

- **FLOPs per cycle:** Normalized performance — e.g., ≈ 2.33 FLOPs per cycle (if 7 FLOPs per iteration over 3 cycles per iteration).

These metrics better represent efficiency and cost on modern CPUs, especially for tight, low-FLOP kernels like this one.

In addition to standard FLOPs/s, the following metrics were calculated:

- **FLOPs per cycle:**

$$\frac{7 \text{ Flops per iteration}}{3.02 \text{ cycles per iteration}} \approx 2.32 \text{ Flop/cycle}$$

- **Iterations per second:**

$$\frac{1}{1.257 * 10^{-9}} \approx 7.955 \times 10^{8} \text{ iterations/second}$$

- **Latency per operation:**

$$\frac{1.257 * 10^{-9}}{7} = 1.796 \times 10^{-10} \text{ sec/op} = 0.1796 \text{ ns/op}$$

4. Turbo Mode and Effective Frequency Estimation (30 Credits)

Experiment Setup

To explore the effect of dynamic clock scaling, I reran the benchmark without fixing the CPU frequency. Instead, I used SLURM's performance governor to allow the CPU to boost to its maximum available frequency:

```
srun --cpu-freq=performance ./pi_bench
```

N = 163840 Result = 3.1415926588 Runtime = 0.000141 seconds

Output

Threshold = 10^{-4} s

```
pi  3.1415926588 with N = 163840
Runtime= 0.000141 seconds
Time per iteration: 0.861 ns/iter
```

Frequency Estimation

Assuming the performance scales linearly with clock speed, the actual frequency during execution can be estimated by comparing time per iteration between fixed and turbo modes:

$$f_{\text{turbo}} = \frac{t_{\text{fixed}}}{t_{\text{turbo}}} \times 2.4 \,\text{GHz}$$

Using:

- $T_{\text{fixed}} = 1.257$ ns
- $T_{\text{turbo}} = 0.861$ ns

$$f_{\text{turbo}} = \frac{1.257}{0.861} \times 2.4 \approx 3.504 \,\text{GHz}$$

Discussion

The calculated frequency matches perfectly with the documented maximum turbo frequency of 3.5 GHz for Fritz.

In turbo mode, the CPU completes each iteration faster, so more iterations are needed before the total runtime crosses the 10^{-4} s threshold. This is why N = 163,840 in turbo mode versus N = 81,920 at fixed 2.4 GHz, it is expected behavior, not an error.

SLURM Job Script Setup

To automate the execution of the benchmark for both fixed-frequency and turbo-mode configurations, the following SLURM job script was created and submitted using `sbatch`:

```
#!/bin/bash -l
#
#SBATCH --nodes=1
#SBATCH --time=00:08:00
#SBATCH --job-name=INTEGRATE
#SBATCH --output=pi_bench.out
#SBATCH --export=NONE

unset SLURM_EXPORT_ENV

module load intel

# Compile
icx -O3 -xHost -o pi_bench pi_bench.c -lm

# Run with fixed CPU frequency
echo "Running with fixed CPU frequency (2.4 GHz)..."
srun --cpu-freq=2400000-2400000:performance ./pi_bench

# Run with turbo (performance governor)
echo "Running with performance governor..."
srun --cpu-freq=performance ./pi_bench
```

Listing 1: SLURM Job Script: job.sh

This script ensures reproducibility by compiling the code within the job environment, and uses `srun` with the appropriate `-cpu-freq` options to differentiate between fixed and dynamic frequency scaling. Output is automatically saved to `pi_bench.out`.